# AI Risk Case Study: Agentic Refund Application

## Independent Risk Assessment, Testing Framework and Pre-Production Acceptance Criteria

**Document Owner:** Dr Oxana Samko

**Document type:** AI Risk Management and Assurance Case Study
**System under assessment:** Agentic AI Refund Application
**Lifecycle stage:** Development, integration testing and pre-production validation
**Primary objective:** Establish whether the application can be safely approved for production deployment
**Risk owners:** Business/Product Owner, AI Risk Owner, Technology Owner, Information Security and Compliance
**Decision outcomes:** PASS, CONDITIONAL PASS where permitted, or FAIL

---

## 1. Use Case — Short Description

The Agentic Refund Application uses a large language model (LLM) to interpret customer refund requests, retrieve relevant order and policy information, determine the next appropriate steps, and interact with backend services through tools or APIs.

Depending on its assigned permissions, the agent may recommend a refund, request human approval, or initiate and execute a refund transaction.

A typical workflow consists of:

1. The customer submits a refund request through a digital channel.
2. The AI agent interprets the request and retrieves customer, order and refund-policy information.
3. The agent determines whether additional information or clarification is required.
4. The application validates eligibility, customer authorisation, refund amount and transaction status.
5. The agent recommends an action or requests approval, depending on its authority.
6. The refund service executes an authorised transaction.
7. The application updates the relevant records, communicates the outcome to the customer and records audit evidence.

The application combines probabilistic AI behaviour with deterministic business processes and financial transactions. Its risk profile therefore extends beyond the accuracy of the LLM response to include agent autonomy, tool execution, transaction integrity, cybersecurity, privacy, customer outcomes and operational resilience.

**Primary risk event:** The AI agent promises, approves, initiates or executes an incorrect, unauthorised, excessive or duplicate refund.

**Core risk management principle:** The LLM may interpret requests and recommend actions, but it must not be the sole authority for determining financial eligibility or authorising a refund. Mandatory controls must be enforced independently by the application and backend services.

## 1.1 Assessment assumptions

This case study assumes that the agent can access order information and invoke at least some backend tools. Direct payment execution may be enabled or may require human approval.

The assessment must be tailored to the actual architecture. Risks involving autonomous execution, multiple agents, external payment providers, retrieval-augmented generation (RAG), or cross-border transactions should be marked not applicable only where the relevant functionality is demonstrably absent.

---

# 2. Risk Identification

## 2.1 Risk assessment methodology

Risks are identified across the complete application lifecycle and transaction journey, rather than being limited to model accuracy.

The assessment covers:

- AI model behaviour and decision quality.
- Agent autonomy, orchestration and tool use.
- Financial transaction and business process integrity.
- Cybersecurity and adversarial threats.
- Data quality, retrieval and policy governance.
- Technology resilience and operational controls.
- Legal, regulatory, conduct and customer protection.
- Third-party dependencies and AI governance.

The following inventory defines the principal risks, the potential consequences and the corresponding testing scope.

Priority classifications are preliminary and must be confirmed through the organisation's risk methodology.

- **Critical:** Potential for unauthorised financial execution, major data compromise or systemic bypass of mandatory controls.
- **High:** Potential for material financial loss, significant customer harm, security incidents or regulatory breaches.
- **Medium:** Material but generally more contained operational, service-quality or governance impact.

## 2.2 AI model and decision-making risks

| ID | Risk | Potential impact | Priority |
|---|---|---|---|
| AI-01 | Misinterpretation of customer intent | Incorrect refund decision or action | High |
| AI-02 | Incorrect order identification | Refund applied to the wrong order | Critical |
| AI-03 | Hallucinated refund policy or exception | Unauthorised commitment or incorrect eligibility decision | High |
| AI-04 | Loss of conversation context | Incorrect action after context truncation or conversation changes | High |
| AI-05 | Cross-session or cross-customer context contamination | Disclosure or incorrect transaction | Critical |
| AI-06 | Poor uncertainty handling | Action taken despite insufficient information | High |
| AI-07 | Inconsistent outputs across repeated executions | Unpredictable decisions and control failures | High |
| AI-08 | Bias or unequal treatment | Unfair or discriminatory customer outcomes | High |
| AI-09 | Multilingual or culturally specific interpretation errors | Incorrect treatment of customer requests | High |
| AI-10 | Reliance on model-generated explanations or reasoning | Unsupported decisions accepted as valid | High |
| AI-11 | False refusal of legitimate refunds | Customer detriment, complaints and loss of trust | High |
| AI-12 | Model drift or provider-side changes | Degraded performance after approval | High |

**Testing focus:** Test ambiguous and contradictory requests, multiple orders in one conversation, context changes, unsupported policy claims, repeated model runs, multilingual inputs, legitimate refund requests and inappropriate refusals. Verify that uncertain or unsupported decisions trigger clarification or escalation rather than financial execution.

## 2.3 Agent autonomy, orchestration and tool-use risks

| ID | Risk | Potential impact | Priority |
|---|---|---|---|
| AG-01 | Excessive autonomy | Agent performs actions beyond its mandate | Critical |
| AG-02 | Unsafe multi-step action sequences | An incorrect plan results in a financial transaction | Critical |
| AG-03 | Tool chaining bypasses individual controls | Combined actions produce an unauthorised outcome | Critical |
| AG-04 | Confused-deputy behaviour | Agent misuses privileged backend access | Critical |
| AG-05 | Incorrect tool arguments | Wrong order, amount, currency or payment destination | Critical |
| AG-06 | Unsafe delegation between agents | Downstream component performs an unauthorised action | High |
| AG-07 | Agent retries or changes tools after denial | Refusal or policy restrictions are circumvented | Critical |
| AG-08 | Human approval bypass or reuse | Unapproved refund execution | Critical |
| AG-09 | Excessive service permissions | Unnecessary access or transaction authority | Critical |
| AG-10 | Time-of-check/time-of-use failure | Eligibility or authorisation becomes invalid | Critical |

| | | | |
|---|---|---|---|
| | | before execution | |
| AG-11 | Unbounded agent loops or tool calls | Resource exhaustion or unintended repeated actions | High |
| AG-12 | Failure to terminate or escalate | Agent continues acting when it should stop | High |

**Testing focus:** Execute complete agent workflows, not merely isolated prompts or tool calls. Test denied actions, alternative tool paths, repeated requests, changes in order state between validation and execution, approval expiry, tool permissions and agent termination. Confirm that every transaction is checked by a trusted enforcement layer immediately before execution.

# 2.4 Financial transaction and business process risks

| ID | Risk | Potential impact | Priority |
|---|---|---|---|
| FIN-01 | Refund sent to the wrong payment instrument | Financial loss or customer harm | Critical |
| FIN-02 | Incorrect currency conversion or rounding | Incorrect refund value | High |
| FIN-03 | Incorrect tax, shipping, discount or fee calculation | Overpayment or underpayment | High |
| FIN-04 | Cumulative over-refunding | Refund exceeds total entitlement | Critical |
| FIN-05 | Refund conflicts with a chargeback or payment dispute | Double recovery or financial loss | High |
| FIN-06 | Invalid transaction-state transition | Refund against a cancelled, reversed or ineligible transaction | High |
| FIN-07 | Concurrent refund requests or race conditions | Multiple refunds against the same entitlement | Critical |
| FIN-08 | Payment, order and ledger reconciliation failure | Incorrect financial records | Critical |
| FIN-09 | Partial failure across distributed services | Inconsistent transaction state | Critical |
| FIN-10 | Incorrect handling of discounts, vouchers or loyalty points | Incorrect reimbursement or accounting | High |
| FIN-11 | Refund-limit abuse across orders or accounts | Cumulative financial loss | High |
| FIN-12 | Incorrect pending, completed, rejected or reversed status | Misleading customer communication and operational errors | High |
| FIN-13 | Refund after the permitted eligibility period | Unauthorised payment | High |
| FIN-14 | Incorrect treatment of partial or item-level refunds | Incorrect refund allocation | High |
| FIN-15 | Incorrect recovery following timeout or uncertain payment status | Duplicate execution | Critical |

**Testing focus:** Test financial calculations, payment destinations, transaction-state rules, partial refunds, multiple refunds, concurrent execution, timeout recovery and reconciliation. Verify actual payment and ledger states rather than relying on the assistant's reported result.

# 2.5 Cybersecurity, privacy and adversarial AI risks

| ID | Risk | Potential impact | Priority |
|---|---|---|---|
| SEC-01 | Direct or indirect prompt injection | Unauthorised actions or policy bypass | Critical |

| | | | |
|---|---|---|---|
| SEC-02 | Knowledge-base or retrieved-data poisoning | Incorrect decisions based on manipulated information | High |
| SEC-03 | Customer account takeover | Unauthorised refund or data access | Critical |
| SEC-04 | Insecure direct object references | Access to another customer's order | Critical |
| SEC-05 | Sensitive information leakage through prompts, logs or traces | Privacy breach | High |
| SEC-06 | Exposure of API keys, tokens or credentials | Unauthorised system access | Critical |
| SEC-07 | Malicious or malformed tool outputs | Unsafe subsequent agent behaviour | High |
| SEC-08 | Denial of service or resource exhaustion | Service disruption and financial exposure | High |
| SEC-09 | Insecure external model or tool integration | Compromise of data or actions | High |
| SEC-10 | Failure to delete or retain data appropriately | Privacy and compliance violations | High |
| SEC-11 | Unauthorised provider use or reuse of customer data | Confidentiality and regulatory risk | High |
| SEC-12 | Insufficient tenant or business-unit isolation | Cross-customer or cross-organisation disclosure | Critical |
| SEC-13 | Weak authentication, session management or authorisation | Unauthorised actions | Critical |

**Testing focus:** Perform threat-led security testing, including prompt injection through customer messages and retrieved documents, cross-account access attempts, session manipulation, secret exposure checks, hostile tool outputs, rate-limit bypass and tenant-isolation testing. Verify access control at the backend, independent of the LLM.

## 2.6 Data quality, retrieval and policy governance risks

| ID | Risk | Potential impact | Priority |
|---|---|---|---|
| DAT-01 | Outdated refund policy | Incorrect eligibility decision | High |
| DAT-02 | Conflicting policy sources | Inconsistent refund decisions | High |
| DAT-03 | Missing, duplicated, stale or inconsistent order data | Incorrect action | High |
| DAT-04 | Incorrect retrieval ranking or missing evidence | Unsupported decision | High |
| DAT-05 | Gaps in policy coverage | Improvised or unsupported decisions | High |
| DAT-06 | Failure to apply jurisdiction-specific rules | Legal or customer harm | High |
| DAT-07 | Policy updates introduce regressions | Previously correct behaviour deteriorates | High |
| DAT-08 | Retrieval crosses customer or tenant boundaries | Data exposure | Critical |
| DAT-09 | Missing data provenance or version information | Inability to verify decision basis | High |
| DAT-10 | Incorrect effective dates or policy precedence | Application of invalid rules | High |

**Testing focus:** Test retrieval accuracy, source precedence, policy effective dates, missing evidence, conflicting documents, customer isolation and jurisdictional differences. Verify that each decision uses the correct approved policy version and that unsupported cases are escalated.

## 2.7 Technology resilience and operational risks

| ID | Risk | Potential impact | Priority |
|---|---|---|---|
| OPS-01 | Model or provider outage | Interrupted or unsafe processing | High |
| OPS-02 | Payment gateway outage | Uncertain transaction outcomes | Critical |
| OPS-03 | Excessive latency | Poor service or abandoned transactions | Medium |
| OPS-04 | Cascading failures or retry storms | Widespread service disruption | High |
| OPS-05 | Inadequate observability | Failure to detect or investigate incidents | High |
| OPS-06 | Missing emergency shutdown capability | Continued unsafe execution | Critical |
| OPS-07 | Inadequate disaster recovery | Loss or duplication of transaction state | High |
| OPS-08 | Uncontrolled configuration changes | Unapproved behaviour in production | High |
| OPS-09 | Uncontrolled inference and tool costs | Financial exposure or service degradation | Medium |
| OPS-10 | Inadequate monitoring and alerting | Delayed detection of harmful activity | High |
| OPS-11 | Incomplete backup and restoration controls | Loss of audit or transaction evidence | High |
| OPS-12 | Inadequate capacity management | Failures during peak demand | High |

**Testing focus:** Conduct load, resilience, fault-injection and recovery tests. Simulate model outages, slow responses, payment timeouts, database failures, deployment rollback and emergency disablement. Verify that failures do not result in duplicate transactions or unsupported success messages.

## 2.8 Legal, regulatory, conduct and customer protection risks

| ID | Risk | Potential impact | Priority |
|---|---|---|---|
| REG-01 | Failure to honour applicable statutory refund rights | Regulatory exposure and customer harm | High |
| REG-02 | Misleading statements about refund entitlement or timing | Customer detriment | High |
| REG-03 | Failure to recognise or escalate complaints | Unresolved customer harm | High |
| REG-04 | Inadequate human review, correction or redress | Customers cannot resolve incorrect decisions | High |
| REG-05 | Incomplete audit trail | Weak accountability and investigation capability | High |
| REG-06 | Non-compliance with applicable privacy obligations | Legal and reputational exposure | High |
| REG-07 | Accessibility barriers | Unequal access to refund services | Medium |
| REG-08 | Inadequate disclosure of AI involvement where required | Transparency and compliance risk | Medium |
| REG-09 | Unclear accountability for AI-generated commitments | Unresolved liability and remediation | High |
| REG-10 | Inconsistent treatment across customer | Unequal outcomes or policy | High |

| | | | |
|---|---|---|---|
| | channels | breaches | |

**Testing focus:** Validate applicable consumer and privacy requirements, communication accuracy, complaints handling, escalation, accessibility, auditability and human redress. Obtain compliance review for the actual jurisdictions and business model.

## 2.9 Third-party, governance and independent assurance risks

| ID | Risk | Potential impact | Priority |
|---|---|---|---|
| GOV-01 | Unapproved model or AI provider | Use of components outside risk appetite | High |
| GOV-02 | Uncontrolled third-party model changes | Behavioural regression | High |
| GOV-03 | Insufficient supplier security or resilience assurance | Unmanaged external dependency risk | High |
| GOV-04 | Incomplete AI system and dependency inventory | Unidentified exposure | Medium |
| GOV-05 | Inadequate segregation of duties | Unauthorised release or self-approval | High |
| GOV-06 | Weak AI incident management | Delayed containment and remediation | High |
| GOV-07 | Unclear risk ownership | Unmanaged residual risk | High |
| GOV-08 | Insufficient independent validation | Undetected design or testing gaps | High |
| GOV-09 | Inadequate change management | Unassessed changes reach production | High |
| GOV-10 | Inadequate documentation of limitations | Unsafe deployment assumptions | High |

**Testing focus:** Review approved model and supplier inventories, version control, change approvals, incident procedures, independent assurance evidence and accountable risk ownership. Verify that material changes trigger appropriate regression and re-approval.

---

# 3. Testing Approach — Illustrative Example for the Agentic Refund Application

## 3.1 Purpose

This section illustrates how developers can perform a risk assessment for the Agentic Refund Application using a practical evaluation workflow.

The objective is to demonstrate how to translate an identified risk into a testable scenario, define the expected behaviour, evaluate the observed results and determine whether the risk is adequately controlled.

The approach is intended as a starting point. Developers should adapt the testing scope, metrics, datasets and acceptance thresholds to the application's architecture, autonomy level, business requirements and risk appetite.

The example focuses on four evaluation reports that help distinguish between overall AI quality, violations of safety requirements, the effectiveness of an LLM-based evaluator, and differences between development and independent test data.

## 3.2 Step 1 — Define the risk scenario and expected behaviour

Start with a specific risk scenario rather than a general objective such as "the model should be accurate."

For example:

**Risk scenario:** A customer asks for a refund for an ineligible order. The AI agent incorrectly tells the customer that the refund is approved or attempts to initiate a refund transaction.

Define the expected behaviour before running the test:

- The application identifies that the refund is ineligible under the applicable policy.
- The agent does not promise an unauthorised refund.
- Any attempted refund action is rejected by the application's independent controls.
- The customer receives an accurate explanation or is referred for further review, where appropriate.
- The outcome is recorded for subsequent investigation.

Developers can then create test cases covering different phrasings, missing information, conflicting context and attempts to influence the agent.

The same method can be applied to other scenarios, such as a wrong-order refund, an excessive refund amount, a duplicate request or an unauthorised tool call.

## 3.3 Step 2 — Run the evaluation and generate illustrative reports

For this example, developers can use a fixed set of recorded LLM responses and generate four reports. Using recorded responses makes the evaluation repeatable and helps distinguish changes in scoring logic from changes in model behaviour.

### Report 1: Overall score and category breakdown

Calculate an overall response-quality score and break it down by relevant categories, such as:

- Eligible refund requests.
- Ineligible refund requests.
- Ambiguous requests.
- Incorrect order references.
- Policy compliance.
- Adversarial requests.

**Risk assessment purpose:** Determine whether an aggregate result conceals poor performance in a high-risk category.

For example, a high overall score may coexist with repeated failures on ineligible refunds. Developers should investigate the category-level results rather than relying exclusively on the average.

## Report 2: Soft thresholds versus hard invariants

Separate probabilistic quality measures from safety requirements.

A soft metric might measure whether the agent correctly explains the refund policy. A hard invariant might require that no refund is executed without valid authorisation.

The report should distinguish between:

1. An incorrect answer that does not trigger an action.
2. An unsafe recommendation or tool request that is blocked by the application.
3. An unsafe refund action that is successfully executed.

**Risk assessment purpose:** Understand both the model's behavioural weaknesses and the effectiveness of the controls that contain them.

For this example, a violation of a critical financial control should trigger investigation and block the release decision until the issue is resolved. A high overall score must not compensate for an unsafe transaction.

## Report 3: Precision and recall of the LLM judge for dangerous behaviour

If an LLM judge is used to identify prohibited refund promises or other unsafe responses, compare its classifications with independently human-labelled examples.

Measure:

- **Precision:** Of the responses flagged as dangerous, what proportion are actually dangerous?
- **Recall:** Of all genuinely dangerous responses, what proportion does the judge identify?
- False positives and false negatives.
- The number and type of examples evaluated.

**Risk assessment purpose:** Determine whether the automated evaluator is reliable enough to support the assessment.

For example, a judge may agree with human reviewers on most responses but still miss a meaningful proportion of prohibited refund promises. Developers should examine those missed cases and decide whether the judge can be used for the intended evaluation purpose.

The judge is an evaluation aid, not a control that authorises or prevents financial execution. The application must independently enforce transaction restrictions.

## Report 4: Working set versus independent holdout set

Evaluate the application using a working dataset during development and a separate holdout dataset for a more independent assessment.

The working set can be used to identify failures, improve prompts and refine the evaluation approach. The holdout set should remain separate from those development activities until the evaluation is ready.

Compare overall and category-level results across the two datasets.

**Risk assessment purpose:** Identify potential overfitting to known examples and assess whether the observed performance generalises to previously unseen cases.

If holdout performance is materially worse, investigate the causes before making a release decision. Depending on the findings, developers may need additional test cases, changes to the application or a new independent holdout set.

## 3.4 Step 3 — Verify the actual application outcome

Recorded LLM responses are useful for reproducible evaluation, but they cannot demonstrate that the complete refund workflow is safe.

For selected high-risk scenarios, developers should also execute integration or end-to-end tests in a controlled environment.

For example, submit a request that attempts to refund the wrong order and verify:

- Which order the agent identifies.
- Which tool calls it generates.
- Whether the backend independently checks order ownership and eligibility.
- Whether a payment transaction is actually initiated.
- Whether the order, payment and ledger records remain correct.
- Whether the customer receives an accurate response.

The test passes only when the observed outcome meets the predefined expected behaviour. A response stating that the refund was rejected is insufficient if the backend nevertheless processed it.

## 3.5 Step 4 — Interpret the results and decide what happens next

Use the reports and execution evidence to classify findings and determine the next action.

| Illustrative finding | Risk assessment response |
|---|---|
| High overall score, but unsafe refund promises remain | Investigate the dangerous category and improve the model or application controls |
| LLM judge misses dangerous responses | Review false negatives, improve evaluation criteria and consider additional human review |
| Working-set results are strong but holdout results deteriorate | Investigate generalisation and expand independent testing |
| Agent requests an incorrect refund, but the backend blocks it | Record a model-quality failure and verify that the protective control operates reliably |
| Incorrect refund is actually executed | Treat as a critical failure, investigate the root cause and remediate before release |

| | |
|---|---|
| Results are inconsistent or insufficient to support a conclusion | Gather more evidence or improve the test design before making the decision |

These examples illustrate how to interpret evidence. The final severity classification and release implications should follow the organisation's approved risk framework.

## 3.6 Step 5 — Document the assessment

For each material risk scenario, record:

- Risk ID and description.
- Test objective and scenario.
- Expected behaviour and acceptance rule.
- Test dataset and relevant system versions.
- Observed model response and tool activity.
- Actual backend outcome, where applicable.
- Evaluation metrics and supporting evidence.
- Pass/fail result and any limitations.
- Identified control weaknesses and remediation actions.
- Residual risk and accountable owner.

This creates a traceable link between the identified risk, the test performed, the evidence collected and the eventual release decision.

## 3.7 How to use this approach

The four reports are an illustrative evaluation method for this refund use case, not a universal requirement for all AI systems. Developers should select the reports and metrics that provide meaningful evidence for the risks identified in their own assessment.

For an agentic refund application, the central question is not simply whether the LLM produces a good response. It is whether the complete system consistently prevents inappropriate financial actions and produces acceptable customer outcomes.

The results of this assessment should feed into the broader pre-production testing strategy and risk acceptance process. Passing these illustrative evaluations alone does not establish that the application is ready for production.

---

# 4. Required Pre-Production Testing Strategy

The testing strategy must combine component testing, AI-specific evaluation, adversarial testing and end-to-end financial transaction validation.

## 4.1 Unit and deterministic control testing

**Objective:** Verify that mandatory business and safety rules are enforced independently of model behaviour.

Required tests:

- Refund eligibility and policy calculations.
- Order ownership and customer authorisation.
- Maximum refundable amount and cumulative refund limits.
- Currency precision, tax and partial-refund calculations.
- Tool argument validation and schema enforcement.
- Approval expiry and segregation of duties.
- Transaction idempotency and duplicate prevention.
- Valid and invalid state transitions.
- Access control for every backend endpoint.

**Acceptance evidence:** Automated test results, code-level control coverage, negative test cases and proof that invalid requests are rejected before financial execution.

## 4.2 LLM evaluation and model validation

**Objective:** Establish the accuracy, consistency and limitations of model-driven behaviour.

Required tests:

- Intent classification and order identification.
- Correct approval, refusal and clarification behaviour.
- Policy grounding and handling of missing information.
- Hallucination and unsupported promise detection.
- Multilingual and ambiguous requests.
- Repeated-run consistency for high-impact cases.
- False approvals and false denials.
- Performance by category and risk severity.
- Independent holdout evaluation.
- LLM judge validation against human labels.

**Acceptance evidence:** Four evaluation reports, confusion matrices for dangerous behaviour, category-level metrics, holdout comparisons and documented limitations.

## 4.3 Agent and tool integration testing

**Objective:** Establish that the agent cannot execute an unsafe plan or exceed its permissions.

Required tests:

- Correct tool selection and argument generation.
- Tool calls against the wrong order or customer.
- Repeated tool calls and retry behaviour.
- Alternative tool paths after denial.
- Multi-step execution and tool chaining.
- Changes in eligibility between decision and execution.
- Human approval enforcement.
- Least-privilege service permissions.
- Tool failure, malformed output and unexpected response handling.

**Acceptance evidence:** End-to-end execution traces, tool-call records, authorisation results and backend state verification.

## 4.4 Financial transaction and reconciliation testing

**Objective:** Verify financial correctness across the entire transaction lifecycle.

Required tests:

- Full and partial refunds.
- Multiple refunds against one order.
- Concurrent requests.
- Duplicate submissions and idempotent retries.
- Timeouts with uncertain payment status.
- Payment success followed by database failure.
- Database success followed by notification failure.
- Reconciliation of order, payment provider and ledger records.
- Chargebacks, cancellations, reversals and disputed transactions.
- Recovery and compensation procedures for inconsistent states.

**Acceptance evidence:** Sandbox transaction logs, ledger reconciliation, payment state transitions and demonstrated recovery without duplicate financial execution.

## 4.5 Security and adversarial testing

**Objective:** Identify attempts to manipulate the agent or bypass the application security boundary.

Required tests:

- Direct and indirect prompt injection.
- Malicious instructions in order notes, documents and retrieved content.
- Cross-customer access and session manipulation.
- Authentication and authorisation bypass attempts.
- Sensitive-data leakage through responses, logs and traces.
- Credential exposure and insecure integration.
- Tool-output manipulation.
- Excessive tool calls, rate-limit bypass and denial-of-service scenarios.
- Tenant isolation and retrieval isolation.

**Acceptance evidence:** Threat model, security test results, vulnerability findings, remediation evidence and confirmation that critical attacks cannot cause unauthorised transactions.

## 4.6 Data and policy testing

**Objective:** Verify that the agent acts on accurate, current and authoritative information.

Required tests:

- Outdated and conflicting policies.
- Missing, duplicated and inconsistent order data.
- Incorrect retrieval ranking.
- Policy effective dates and jurisdictional precedence.
- Malicious or unapproved knowledge-base updates.
- Missing policy coverage.
- Data provenance and version tracking.
- Regression testing following policy changes.

**Acceptance evidence:** Retrieval and policy test results, approved source inventory, policy version records and regression reports.

## 4.7 Performance, resilience and disaster recovery

**Objective:** Demonstrate safe behaviour under normal load, peak load and service failure.

Required tests:

- Expected and peak transaction volumes.
- Latency and throughput against approved service objectives.
- Model provider outages.

- Payment gateway outages and timeouts.
- Database and network failures.
- Retry storms and cascading failures.
- Backup and restoration.
- Transaction recovery after interruption.
- Emergency disablement of AI-driven actions.
- Rollback to an approved application and model version.

**Acceptance evidence:** Load-test results, fault-injection records, recovery results, service-level measurements and verified shutdown procedures.

## 4.8 Compliance, customer outcomes and governance testing

**Objective:** Verify that the service operates within applicable legal and business obligations.

Required tests:

- Applicable refund and consumer rights.
- Customer communications and refund-status accuracy.
- Complaints, escalation and human review.
- Customer correction and redress.
- Privacy, retention and deletion.
- Accessibility and supported customer channels.
- Audit evidence and decision provenance.
- Third-party and supplier assurance.
- Change management, approval responsibilities and incident procedures.

**Acceptance evidence:** Compliance mapping, approved customer journeys, audit samples, supplier evidence, accountable owners and documented residual risks.

## 4.9 Independent validation

An independent reviewer should challenge the completeness of the risk inventory, test design, data representativeness, hard-invariant enforcement and interpretation of results.

The reviewer must verify that:

- Test data has not been improperly reused for tuning.
- The holdout is genuinely independent.
- Critical failures are not concealed by aggregate metrics.
- Automated evaluators are calibrated and their limitations documented.

- Financial outcomes have been independently checked against backend records.
- Findings are traceable to remediation evidence and closure decisions.

---

# 5. Risk Acceptance Criteria

## 5.1 Mandatory hard-control criteria

The following are non-negotiable production gates.

| ID | Acceptance criterion | Pass requirement |
|---|---|---|
| RAC-01 | Refund eligibility and customer authorisation | All mandatory negative tests reject ineligible or unauthorised requests |
| RAC-02 | Order ownership and tenant isolation | Zero observed unauthorised cross-customer or cross-tenant transactions |
| RAC-03 | Refund amount and destination | Every tested transaction respects independently validated amount and destination rules |
| RAC-04 | Duplicate and cumulative refunds | No duplicate or excessive refunds in the approved retry, concurrency and cumulative-limit tests |
| RAC-05 | Tool permissions and agent authority | All tested attempts to exceed the agent's authority are blocked |
| RAC-06 | Transaction state and reconciliation | No unresolved critical inconsistency in the tested financial workflows |
| RAC-07 | Privacy and security | No unresolved critical vulnerability or demonstrated critical control bypass |
| RAC-08 | Safe failure and shutdown | The system fails safely and AI-driven execution can be disabled as designed |
| RAC-09 | Auditability | Required transaction, authorisation and decision evidence is available |
| RAC-10 | Critical scenario coverage | All approved critical scenarios have executed successfully |

Zero observed violations applies to the defined acceptance test suite; it does not imply zero risk in production.

## 5.2 Model quality criteria

Initial proposed thresholds are shown below. They must be approved by the business and risk owners before testing begins and calibrated to the expected risk, sample sizes and consequences of error.

| ID | Metric | Proposed threshold |
|---|---|---|
| RAC-11 | Overall response correctness | At least 95% |
| RAC-12 | Refund-policy response correctness | At least 98% |
| RAC-13 | Correct order identification in model-generated requests | At least 99% |
| RAC-14 | Correct refusal of ineligible refunds | At least 99% |
| RAC-15 | Policy-grounded response quality | At least 95% |
| RAC-16 | Ambiguous and unusual request quality | At least 90% |
| RAC-17 | Valid output schema or safe rejection | 100% |

| | | |
|---|---|---|
| RAC-18 | Judge precision for prohibited refund promises | At least 95% |
| RAC-19 | Judge recall for prohibited refund promises | At least 98% |

These proposed thresholds are not universal standards. The risk owner must decide whether they are appropriate for the application and whether the test sample provides sufficient statistical confidence.

For example, 99% measured accuracy on a small test set is not strong evidence of 99% population accuracy. Confidence intervals, class prevalence and the number of dangerous examples must be considered. Zero observed critical failures in a finite test set also does not prove that critical failures are impossible.

**Hard-control failures cannot be offset by passing model-quality metrics.**

## 5.3 Dataset and evaluation criteria

| ID | Acceptance criterion | Pass requirement |
|---|---|---|
| RAC-20 | Independent holdout | Frozen, documented and not used for tuning |
| RAC-21 | Risk category coverage | All approved critical and high-risk categories represented |
| RAC-22 | Human-labelled ground truth | Dangerous-class labels independently reviewed |
| RAC-23 | Evaluation reproducibility | Results can be reproduced from recorded inputs, outputs and versions |
| RAC-24 | Category-level reporting | False approvals and false denials reported separately |
| RAC-25 | Regression testing | Required tests rerun after material model, prompt, policy or tool changes |

## 5.4 Operational and governance criteria

| ID | Acceptance criterion | Pass requirement |
|---|---|---|
| RAC-26 | Service objectives | Approved latency, availability and throughput targets met |
| RAC-27 | Incident detection | Alerts for critical refund anomalies tested |
| RAC-28 | Emergency controls | Kill switch and rollback verified |
| RAC-29 | Compliance | Applicable obligations reviewed and mapped to controls |
| RAC-30 | Defect management | No unresolved critical or unaccepted high-risk findings |
| RAC-31 | Accountability | Required business, technology, security and risk approvals recorded |
| RAC-32 | Residual risk | Documented and formally accepted within approved risk appetite |

---

# 6. Proposed Risk-Based Release Decision Matrix

## 6.1 Release outcomes

| Outcome | Conditions | Decision |
|---|---|---|
| **PASS** | All mandatory hard controls pass; approved model and operational thresholds pass; no unresolved critical or high-risk blockers; required evidence and approvals are complete | Eligible for production deployment |

| | | |
|---|---|---|
| **CONDITIONAL PASS** | Only explicitly permitted, non-critical residual risks remain; effective compensating controls, accountable owners and time-bound remediation plans exist; organisational policy permits conditional approval | Limited or controlled release only after authorised approval |
| **FAIL** | Any critical invariant fails, mandatory control is bypassed, an approved threshold fails, or a critical vulnerability remains unresolved | Release blocked |
| **INCOMPLETE** | Required test evidence, independent review, coverage or approvals are missing | Release blocked pending completion |

Conditional approval must not be used to waive mandatory financial authorisation, privacy, security or other non-negotiable controls. If the organisation does not permit conditional release, the outcome must be FAIL or release blocked until the outstanding items are resolved.

# 6.2 Risk-based decision rules

| Finding | Required action | Release implication |
|---|---|---|
| Incorrect refund executed in a critical test | Disable the relevant execution path, investigate root cause, remediate and rerun affected tests | FAIL |
| Agent bypasses a policy or authorisation control | Correct the enforcement design and independently retest | FAIL |
| Cross-customer data access is possible | Contain the vulnerability, remediate and complete security retesting | FAIL |
| Critical transaction inconsistency or duplicate payment | Remediate transaction handling and verify recovery and reconciliation | FAIL |
| Model quality falls below an approved threshold | Investigate the category-level failure and rerun evaluation after remediation | FAIL |
| Holdout performance materially deteriorates | Investigate overfitting, data shift and evaluation integrity; reassess release readiness | FAIL pending resolution |
| Medium-risk defect with effective controls | Document residual risk, owner, remediation date and monitoring | Conditional approval only if policy permits |
| Required evidence or owner approval is absent | Complete the evidence or approval process | INCOMPLETE |
| All gates pass with acceptable residual risk | Record final assessment and sign-offs | PASS |

# 6.3 Final decision flow

The release decision must follow this sequence:

**1. Scope and evidence complete?**

- No → INCOMPLETE; release blocked.
- Yes → Proceed to hard-control testing.

**2. All mandatory hard invariants and security controls pass?**

- No → FAIL; release blocked.

- Yes → Proceed to AI evaluation.

**3. Model, judge and independent holdout thresholds pass?**

- No → FAIL; release blocked until remediated.
- Yes → Proceed to integration and operational readiness.

**4. Financial reconciliation, resilience, monitoring and shutdown tests pass?**

- No → FAIL; release blocked.
- Yes → Proceed to governance review.

**5. Compliance, independent validation and risk approvals complete?**

- No → INCOMPLETE; release blocked.
- Yes → Assess residual risk.

**6. Residual risk within approved appetite?**

- No → FAIL or remediation required.
- Yes → PASS, subject to formal production approval.

# 6.4 Final risk acceptance statement

Production deployment of the Agentic Refund Application is permitted only when the development and assurance teams demonstrate, through reproducible and risk-based testing, that the end-to-end system operates within approved business, financial, security, privacy, operational and regulatory boundaries.

The LLM must not be the sole authority for refund eligibility or financial execution. Mandatory controls must be independently enforced by the application and backend services.

All critical scenarios must pass; approved AI quality thresholds must be met on appropriately representative and independent evaluation data; critical financial and security controls must be verified; and required business, technology, security, compliance and risk approvals must be recorded.

Any observed violation of a mandatory hard invariant, demonstrated bypass of a critical control, unresolved critical vulnerability, or failure of an approved mandatory acceptance threshold results in **FAIL — RELEASE BLOCKED**.

The final decision must document the evidence reviewed, outstanding limitations, residual risks, compensating controls where permitted, accountable owners and approval rationale. Passing the test suite does not establish that production risk is zero; ongoing monitoring, incident management, regression testing and change control remain mandatory after deployment.

**Final principle:** Production acceptance must be based on the safety and reliability of the entire agentic refund system—not on the apparent quality of its conversational responses alone.